Homework 1: JUnit Testing

İsmail Çetintaşlı, 10002624

Umut SARAÇ //as I know his last name not sure, done alone he did not contribute to the work neither done the homework

Course: SENG 311

Project: Student Management System (Java), taken from https://github.com/Neil-Cardoz/Student-Exception-Java-Lab. I added four methods to `StudentOperations`: `getAverageMarks`, `getTopStudent`, `getStudentsAboveMarks` and `sortByMarks`.

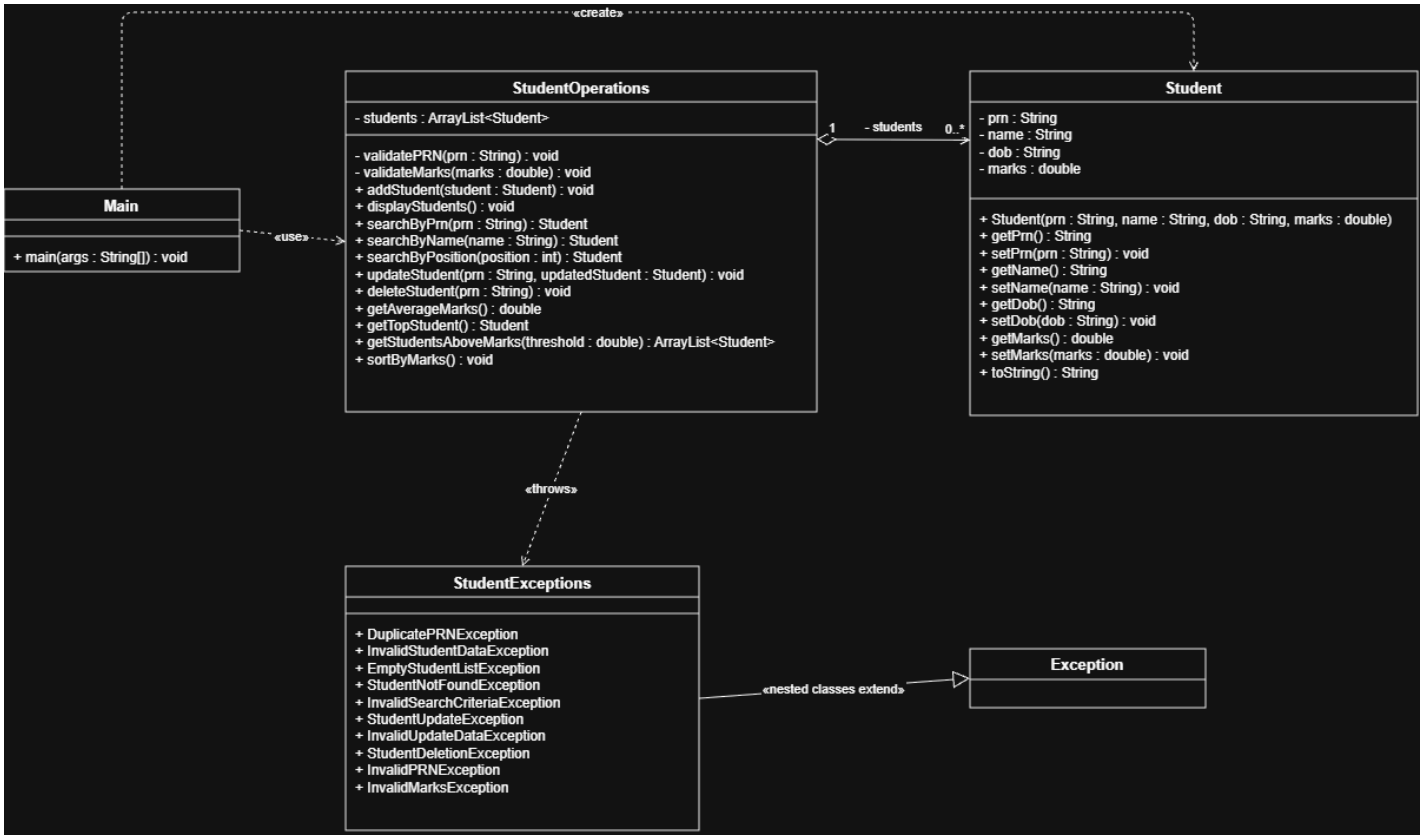
System Overview

The project is a console-based Student Management System written in Java. It keeps students (PRN, name, date of birth, marks) in an ArrayList and offers a simple menu in Main.java.

What it does: add a student, display all students, search by PRN, name or position, update a student, delete a student, and calculate statistics (average marks, top student, students above a given mark, sort by marks).

Rules: a PRN must be exactly 11 digits, marks must be between 0 and 100, and a PRN cannot be added twice. When a rule is broken or a student is not found, the system throws a custom exception defined in StudentExceptions.java (for example DuplicatePRNException, StudentNotFoundException, InvalidStudentDataException).

1. UML Class Diagram



2. Test Code

Assertions Used

The tests use JUnit 5 assertions from org.junit.jupiter.api.Assertions. Test numbers refer to the // comments in the code.

Assertion	What it checks	Used in tests
`assertEquals`	Two values are equal (names, marks, messages, output text)	1, 2, 3, 8, 12, 17, 19, 38
`assertThrows`	The expected exception is thrown	8-12, 14, 16, 20-23, 26-28, 30-32, 34-37
`assertSame`	Same object (not just equal data) is returned	6, 7, 24, 29, 33, 35, 36, 39
`assertNotSame`	Two references are different objects	4, 33
`assertAll`	Several assertions run together and all results are reported	1, 2, 7, 13, 18, 33, 36, 40
`assertTrue`	A condition is true (e.g. message contains text)	11, 15, 18, 21, 35
`assertFalse`	A condition is false (e.g. list does not contain a student)	40
`assertNull`	A value is null	5
`assertNotNull`	A value is not null	33
`assertDoesNotThrow`	No exception is thrown	6, 13
`assertIterableEquals`	Two lists have the same elements in the same order	40

Test 10 is also a parameterized test (@ParameterizedTest with @NullAndEmptySource and @ValueSource) that runs the same invalid-PRN check with many inputs. @BeforeEach creates fresh test data before every test.

Suite 1: ModelAndAddDisplaySuite

StudentModelTest (tests 1-5)

```
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

public class StudentModelTest {

    // 1
    @Test
    void constructorAndGetters_storeAllFields() {
        Student s = new Student("49126067566", "Ismail", "01.01.2006", 80.5);
        assertAll("student fields",
                () -> assertEquals("49126067566", s.getPrn()),
                () -> assertEquals("Ismail", s.getName()),
                () -> assertEquals("01.01.2006", s.getDob()),
                () -> assertEquals(80.5, s.getMarks(), 0.001));
    }

    // 2
    @Test
    void setters_updateAllFields() {
        Student s = new Student("49126067566", "Ismail", "01.01.2006", 80.5);
        s.setPrn("11111111111");
        s.setName("Eren");
        s.setDob("02.02.2007");
        s.setMarks(90.0);

        assertAll("updated fields",
                () -> assertEquals("11111111111", s.getPrn()),
                () -> assertEquals("Eren", s.getName()),
                () -> assertEquals("02.02.2007", s.getDob()),
                () -> assertEquals(90.0, s.getMarks(), 0.001));
```

```
    }

    // 3
    @Test
    void toString_hasExpectedOutput() {
        Student s = new Student("49126067566", "Ismail", "01.01.2006", 80.5);

        assertEquals("PRN: 49126067566, Name: Ismail, DoB: 01.01.2006, Marks: 80.5",
                s.toString());
    }

    // 4
    @Test
    void twoStudentsWithSameData_areDifferentObjects() {
        Student s = new Student("49126067566", "Ismail", "01.01.2006", 80.5);
        Student s1 = new Student("49126067566", "Ismail", "01.01.2006", 80.5);

        assertNotSame(s, s1);
    }

    // 5
    @Test
    void constructor_withNullName_storesNull() {
        Student s = new Student("49126067566", null, "01.01.2006", 80.5);

        assertNull(s.getName());
    }
}
```

AddStudentTest (tests 6-15, test 10 is the parameterized test)

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.NullAndEmptySource;
import org.junit.jupiter.params.provider.ValueSource;

import static org.junit.jupiter.api.Assertions.*;

public class AddStudentTest {
    private StudentOperations operations;

    @BeforeEach
    void setUp() {
        operations = new StudentOperations();
    }

    // 6
    @Test
    void addValidStudent_thenSearchByPrn() {
        Student s = new Student("49126067566", "Ismail", "01.01.2006", 80.5);

        assertDoesNotThrow(() -> operations.addStudent(s));
        assertDoesNotThrow(() -> assertSame(s, operations.searchByPrn("49126067566")));
    }

    // 7
    @Test
    void addTwoStudents_verifyByPosition() throws Exception {
        Student s = new Student("12345678901", "Ismail", "01.01.2006", 80.5);
        Student s1 = new Student("23456789012", "Eren", "01.01.2008", 81.5);

        operations.addStudent(s);
        operations.addStudent(s1);

        assertAll("student positions",
                () -> assertSame(s, operations.searchByPosition(0)),
                () -> assertSame(s1, operations.searchByPosition(1)));
    }

    // 8
    @Test
    void addDuplicatePrn_throwsDuplicatePRNException() throws Exception {
        Student s = new Student("49126067566", "Ismail", "01.01.2006", 80.5);
        Student s1 = new Student("49126067566", "Eren", "01.01.2008", 80.5);
```

```
        operations.addStudent(s);

        StudentExceptions.DuplicatePRNException exception = assertThrows(
                StudentExceptions.DuplicatePRNException.class,
                () -> operations.addStudent(s1));

        assertEquals("Student with PRN " + s.getPrn() + " already exists.", exception.getMessage());
    }

    // 9
    @Test
    void addDuplicatePrn_listSizeAndMarksUnchanged() throws Exception {
        Student s = new Student("49126067566", "Ismail", "01.01.2006", 80.5);
        Student s1 = new Student("49126067566", "Ismail", "01.01.2006", 20.5);

        operations.addStudent(s);

        assertThrows(StudentExceptions.DuplicatePRNException.class, () -> operations.addStudent(s1));
        assertThrows(StudentExceptions.StudentNotFoundException.class, () -> operations.searchByPosition(1));
        assertEquals(80.5, operations.searchByPosition(0).getMarks(), 0.001);
    }

    // 10 (parameterized)
    @ParameterizedTest
    @NullAndEmptySource
    @ValueSource(strings = {"123", "1234567890", "123456789012", "12345abc901", "12345 67890", "
"})
    void addStudent_withInvalidPrn_throwsInvalidStudentDataException(String invalidPrn) {
        Student student = new Student(invalidPrn, "Ismail", "01.01.2006", 85.0);

        assertThrows(StudentExceptions.InvalidStudentDataException.class,
                () -> operations.addStudent(student));
    }

    // 11
    @Test
    void addNegativeMarks_messageContainsExpected() {
        Student student = new Student("12345678901", "Ismail", "01.01.2006", -0.1);

        StudentExceptions.InvalidStudentDataException exception = assertThrows(
                StudentExceptions.InvalidStudentDataException.class,
                () -> operations.addStudent(student));

        assertTrue(exception.getMessage().contains("Invalid marks value"));
    }

    // 12
    @Test
    void addOver100Marks_messageEqualsExpected() {
        Student s = new Student("49126067566", "Ismail", "01.01.2006", 102);

        StudentExceptions.InvalidStudentDataException exception = assertThrows(
                StudentExceptions.InvalidStudentDataException.class,
                () -> operations.addStudent(s));

        assertEquals("Invalid student data: Invalid marks value: " + s.getMarks(), exception.getMessage());
    }

    // 13
    @Test
    void addBoundaryMarks_doesNotThrow() {
        Student sMin = new Student("49126067566", "Ismail", "01.01.2006", 0);
        Student sMax = new Student("12345678901", "Eren", "01.01.2008", 100);

        assertAll("boundary marks",
                () -> assertDoesNotThrow(() -> operations.addStudent(sMin)),
                () -> assertDoesNotThrow(() -> operations.addStudent(sMax)));
    }

    // 14
    @Test
    void addNullStudent_throwsNullPointerException() {
        assertThrows(NullPointerException.class, () -> operations.addStudent(null));
    }
```

```
    // 15
    @Test
    void addStudent_printsSuccessMessage() throws Exception {
        java.io.ByteArrayOutputStream outContent = new java.io.ByteArrayOutputStream();
        java.io.PrintStream originalOut = System.out;
        System.setOut(new java.io.PrintStream(outContent));

        try {
            Student student = new Student("12345678901", "Ismail", "01.01.2006", 85.0);
            operations.addStudent(student);

            assertTrue(outContent.toString().contains("Student added successfully"));
        } finally {
            System.setOut(originalOut);
        }
    }
}
```

DisplayStudentsTest (tests 16-18)

```
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.io.ByteArrayOutputStream;
import java.io.PrintStream;

import static org.junit.jupiter.api.Assertions.*;

public class DisplayStudentsTest {
    private StudentOperations operations;
    private final ByteArrayOutputStream outContent = new ByteArrayOutputStream();
    private final PrintStream originalOut = System.out;

    @BeforeEach
    void setUp() {
        operations = new StudentOperations();
        System.setOut(new PrintStream(outContent));
    }

    @AfterEach
    void restoreStreams() {
        System.setOut(originalOut);
    }

    // 16
    @Test
    void displayStudents_emptyList_throwsException() {
        StudentExceptions.EmptyStudentListException exception = assertThrows(
                StudentExceptions.EmptyStudentListException.class,
                () -> operations.displayStudents());

        assertEquals("No students available in the system.", exception.getMessage());
    }

    // 17
    @Test
    void displayStudents_singleStudent_printsCorrectFormat() throws Exception {
        Student s = new Student("49126067566", "Ismail", "01.01.2006", 80);
        operations.addStudent(s);
        outContent.reset();

        operations.displayStudents();

        String expectedOutput = "Position 0: " + s + System.lineSeparator();
        assertEquals(expectedOutput, outContent.toString());
    }

    // 18
    @Test
    void displayStudents_multipleStudents_printsAllPositions() throws Exception {
        Student s0 = new Student("12345678901", "Ismail", "01.01.2006", 85.0);
        Student s1 = new Student("12345678902", "Eren", "01.01.2008", 90.0);
        Student s2 = new Student("12345678903", "Ahmet", "02.02.2005", 75.0);

        operations.addStudent(s0);
```

```
        operations.addStudent(s1);
        operations.addStudent(s2);
        outContent.reset();

        operations.displayStudents();
        String output = outContent.toString();

        assertAll("multiple student output",
                () -> assertTrue(output.contains("Position 0: " + s0)),
                () -> assertTrue(output.contains("Position 1: " + s1)),
                () -> assertTrue(output.contains("Position 2: " + s2)));
    }
}
```

ModelAndAddDisplaySuite

```
import org.junit.platform.suite.api.SelectClasses;
import org.junit.platform.suite.api.Suite;
import org.junit.platform.suite.api.SuiteDisplayName;

@Suite
@SuiteDisplayName("Suite 1 - Student model, add and display")
@SelectClasses({
        StudentModelTest.class,
        AddStudentTest.class,
        DisplayStudentsTest.class
})
public class ModelAndAddDisplaySuite {
}
```

Suite 2: SearchUpdateDeleteStatsSuite

SearchStudentTest (tests 19-32)

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

public class SearchStudentTest {
    private StudentOperations operations;
    private Student student1;
    private Student student2;
    private Student student3;

    @BeforeEach
    void setUp() throws Exception {
        operations = new StudentOperations();

        student1 = new Student("11111111111", "Ismail", "01.01.2006", 80.0);
        student2 = new Student("22222222222", "Eren", "01.01.2008", 85.0);
        student3 = new Student("33333333333", "Ahmet", "01.01.2005", 90.0);

        operations.addStudent(student1);
        operations.addStudent(student2);
        operations.addStudent(student3);
    }

    // 19
    @Test
    void searchByPrn_lastStudent_returnsCorrectName() throws Exception {
        Student found = operations.searchByPrn("33333333333");

        assertEquals("Ahmet", found.getName());
    }

    // 20
    @Test
    void searchByPrn_validPrnNotInList_throwsStudentNotFoundException() {
        String missingPrn = "44444444444";

        StudentExceptions.StudentNotFoundException exception = assertThrows(
                StudentExceptions.StudentNotFoundException.class,
                () -> operations.searchByPrn(missingPrn));
```

```
        assertEquals("Student not found: PRN: " + missingPrn, exception.getMessage());
    }

    // 21
    @Test
    void searchByPrn_invalidPrnFormat_throwsInvalidSearchCriteriaException() {
        StudentExceptions.InvalidSearchCriteriaException exception = assertThrows(
                StudentExceptions.InvalidSearchCriteriaException.class,
                () -> operations.searchByPrn("123"));

        assertTrue(exception.getMessage().startsWith("Invalid search criteria: Invalid PRN format"));
    }

    // 22
    @Test
    void searchByPrn_nullPrn_throwsInvalidSearchCriteriaException() {
        assertThrows(StudentExceptions.InvalidSearchCriteriaException.class,
                () -> operations.searchByPrn(null));
    }

    // 23
    @Test
    void searchByPrn_emptyRepository_throwsStudentNotFoundException() {
        StudentOperations emptyOperations = new StudentOperations();

        assertThrows(StudentExceptions.StudentNotFoundException.class,
                () -> emptyOperations.searchByPrn("11111111111"));
    }

    // 24
    @Test
    void searchByName_exactName_returnsSameObject() throws Exception {
        Student found = operations.searchByName("Ismail");

        assertSame(student1, found);
    }

    // 25
    @Test
    void searchByName_lowerCase_findsStudentAndChecksPrn() throws Exception {
        Student s4 = new Student("44444444444", "Emre", "01.01.2000", 77);
        operations.addStudent(s4);

        Student found = operations.searchByName("emre");

        assertEquals("44444444444", found.getPrn());
    }

    // 26
    @Test
    void searchByName_nonExistingName_throwsExceptionAndMessage() {
        StudentExceptions.StudentNotFoundException exception = assertThrows(
                StudentExceptions.StudentNotFoundException.class,
                () -> operations.searchByName("Non existing name"));

        assertEquals("Student not found: Name: Non existing name", exception.getMessage());
    }

    // 27
    @Test
    void searchByName_nullName_throwsExceptionAndMessage() {
        StudentExceptions.InvalidSearchCriteriaException exception = assertThrows(
                StudentExceptions.InvalidSearchCriteriaException.class,
                () -> operations.searchByName(null));

        assertEquals("Invalid search criteria: Name cannot be empty", exception.getMessage());
    }

    // 28
    @Test
    void searchByName_partialName_throwsStudentNotFoundException() throws Exception {
        Student s4 = new Student("99999999999", "Ali Haydar", "01.01.2009", 21);
        operations.addStudent(s4);

        assertThrows(StudentExceptions.StudentNotFoundException.class,
```

```
                () -> operations.searchByName("Ali"));
    }

    // 29
    @Test
    void searchByPosition_zero_returnsFirstStudent() throws Exception {
        Student found = operations.searchByPosition(0);

        assertSame(student1, found);
    }

    // 30
    @Test
    void searchByPosition_negative_throwsExceptionAndMessage() {
        StudentExceptions.InvalidSearchCriteriaException exception = assertThrows(
                StudentExceptions.InvalidSearchCriteriaException.class,
                () -> operations.searchByPosition(-1));

        assertEquals("Invalid search criteria: Position cannot be negative", exception.getMessage());
    }

    // 31
    @Test
    void searchByPosition_equalsListSize_throwsExceptionAndMessage() {
        StudentExceptions.StudentNotFoundException exception = assertThrows(
                StudentExceptions.StudentNotFoundException.class,
                () -> operations.searchByPosition(3));

        assertEquals("Student not found: Position: 3", exception.getMessage());
    }

    // 32
    @Test
    void searchByPosition_emptyRepo_throwsException() {
        StudentOperations emptyOperations = new StudentOperations();

        assertThrows(StudentExceptions.StudentNotFoundException.class,
                () -> emptyOperations.searchByPosition(0));
    }
}
```

UpdateDeleteStudentTest (tests 33-37)

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

public class UpdateDeleteStudentTest {
    private StudentOperations operations;
    private Student student1;
    private Student student2;
    private Student student3;

    @BeforeEach
    void setUp() throws Exception {
        operations = new StudentOperations();

        student1 = new Student("11111111111", "Ismail", "01.01.2006", 80.0);
        student2 = new Student("22222222222", "Eren", "01.01.2008", 85.0);
        student3 = new Student("33333333333", "Ahmet", "01.01.2005", 90.0);

        operations.addStudent(student1);
        operations.addStudent(student2);
        operations.addStudent(student3);
    }

    // 33
    @Test
    void updateStudent_existingPrn_replacesStudent() throws Exception {
        Student updated = new Student("22222222222", "Eren Updated", "01.01.2008", 95.0);

        operations.updateStudent("22222222222", updated);

        Student found = operations.searchByPrn("22222222222");
```

```
        assertAll("updated student",
                () -> assertNotNull(found),
                () -> assertSame(updated, found),
                () -> assertNotSame(student2, found),
                () -> assertEquals("Eren Updated", found.getName()),
                () -> assertEquals(95.0, found.getMarks(), 0.001));
    }

    // 34
    @Test
    void updateStudent_nonExistingPrn_throwsStudentUpdateException() {
        Student updated = new Student("44444444444", "Nobody", "01.01.2000", 50.0);

        StudentExceptions.StudentUpdateException exception = assertThrows(
                StudentExceptions.StudentUpdateException.class,
                () -> operations.updateStudent("44444444444", updated));

        assertEquals("Failed to update student: Student with PRN 44444444444 not found",
                exception.getMessage());
    }

    // 35
    @Test
    void updateStudent_invalidMarks_throwsAndKeepsOriginal() throws Exception {
        Student invalid = new Student("22222222222", "Eren", "01.01.2008", 150.0);

        StudentExceptions.InvalidUpdateDataException exception = assertThrows(
                StudentExceptions.InvalidUpdateDataException.class,
                () -> operations.updateStudent("22222222222", invalid));

        assertTrue(exception.getMessage().startsWith("Invalid update data: Invalid marks value"));
        assertSame(student2, operations.searchByPrn("22222222222"));
    }

    // 36
    @Test
    void deleteStudent_existingPrn_removesAndShiftsPositions() throws Exception {
        operations.deleteStudent("22222222222");

        assertAll("after delete",
                () -> assertThrows(StudentExceptions.StudentNotFoundException.class,
                        () -> operations.searchByPrn("22222222222")),
                () -> assertSame(student3, operations.searchByPosition(1)),
                () -> assertThrows(StudentExceptions.StudentNotFoundException.class,
                        () -> operations.searchByPosition(2)));
    }

    // 37
    @Test
    void deleteStudent_nonExistingPrn_throwsStudentDeletionException() {
        StudentExceptions.StudentDeletionException exception = assertThrows(
                StudentExceptions.StudentDeletionException.class,
                () -> operations.deleteStudent("44444444444"));

        assertEquals("Failed to delete student: Student with PRN 44444444444 not found",
                exception.getMessage());
    }
}
```

StatisticsStudentTest (tests 38-40)

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.util.ArrayList;
import java.util.List;

import static org.junit.jupiter.api.Assertions.*;

public class StatisticsStudentTest {
    private StudentOperations operations;
    private Student student1;
    private Student student2;
    private Student student3;

    @BeforeEach
```

```
    void setUp() throws Exception {
        operations = new StudentOperations();

        student1 = new Student("11111111111", "Ismail", "01.01.2006", 80.0);
        student2 = new Student("22222222222", "Eren", "01.01.2008", 85.0);
        student3 = new Student("33333333333", "Ahmet", "01.01.2005", 90.0);

        operations.addStudent(student1);
        operations.addStudent(student2);
        operations.addStudent(student3);
    }

    // 38
    @Test
    void getAverageMarks_threeStudents_returnsAverage() throws Exception {
        assertEquals(85.0, operations.getAverageMarks(), 0.001);
    }

    // 39
    @Test
    void getTopStudent_returnsStudentWithHighestMarks() throws Exception {
        assertSame(student3, operations.getTopStudent());
    }

    // 40
    @Test
    void getStudentsAboveMarks_threshold80_returnsOnlyHigherStudents() throws Exception {
        ArrayList<Student> result = operations.getStudentsAboveMarks(80.0);

        assertAll("above 80",
                () -> assertIterableEquals(List.of(student2, student3), result),
                () -> assertFalse(result.contains(student1)));
    }
}
```

SearchUpdateDeleteStatsSuite

```
import org.junit.platform.suite.api.SelectClasses;
import org.junit.platform.suite.api.Suite;
import org.junit.platform.suite.api.SuiteDisplayName;

@Suite
@SuiteDisplayName("Suite 2 - Search, update, delete and statistics")
@SelectClasses({
        SearchStudentTest.class,
        UpdateDeleteStudentTest.class,
        StatisticsStudentTest.class
})
public class SearchUpdateDeleteStatsSuite {
}
```